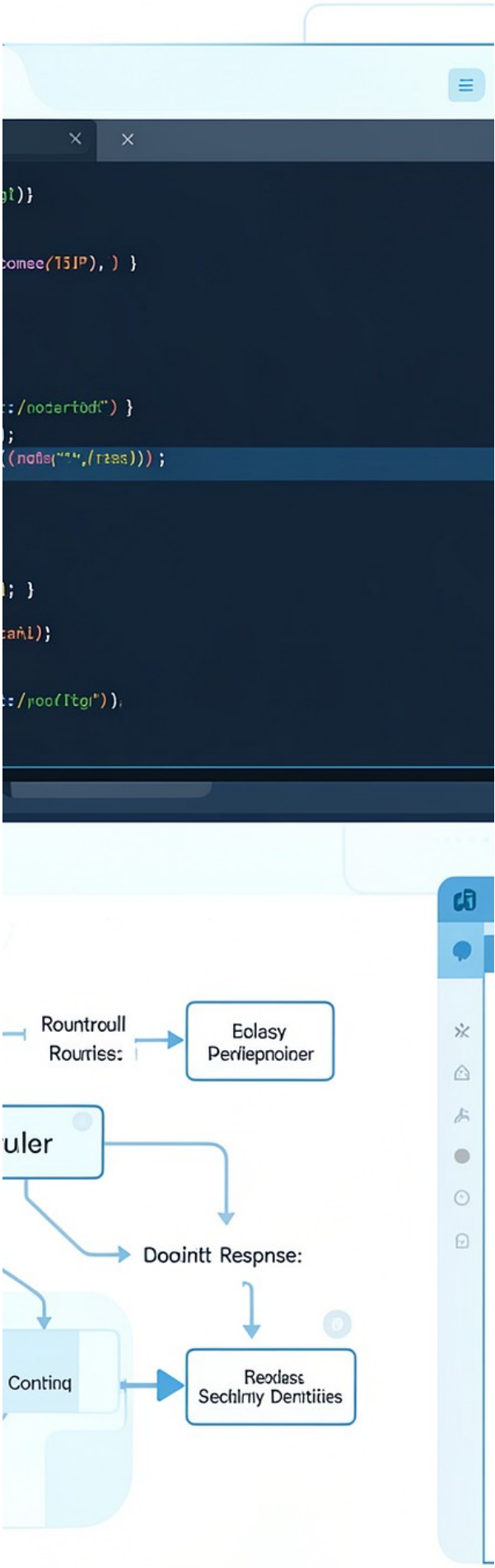
LARAVEL LEARNING MODULE

MATERI 2: ROUTING, CONTROLLER, REQUEST, RESPONSE, DAN BLADE

Memahami Alur Request-Response dalam Laravel

Learning Objectives

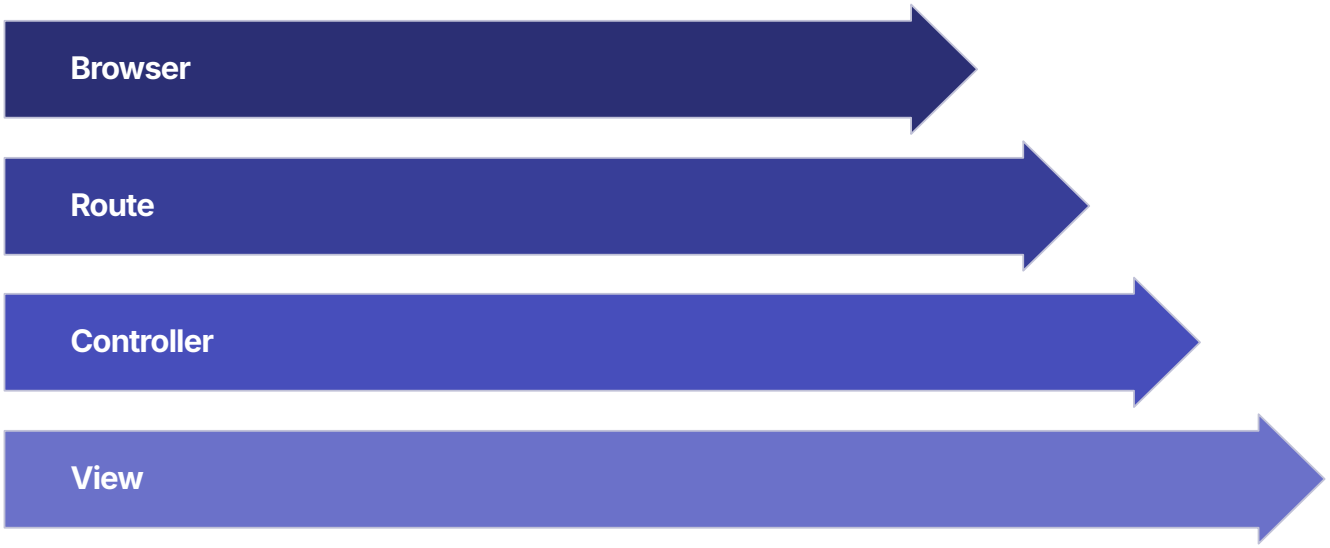
- Membuat dan menggunakan Route
- Membuat Controller dan menghubungkannya
- Memahami Request dan Response
- Menampilkan data dengan Blade Template
- Menggunakan Layout dan Component Blade
- Debugging dengan bantuan AI



Made with GAMMA

Alur Request-Response dalam Laravel

Berikut gambaran sederhana bagaimana permintaan dari browser diproses di Laravel hingga kembali menjadi respons.



Browser

Pengguna mengirim request ke aplikasi melalui halaman web.

Route

Laravel menentukan rute mana yang menangani request tersebut.

Controller

Controller memproses logika aplikasi dan menyiapkan data.

View

Data dirender ke tampilan agar bisa dibaca pengguna.

Response

Hasil akhir dikirim kembali ke browser sebagai respons.

Made with GAMMA

A. ROUTING - Konsep Dasar

Route menentukan URL mana yang akan menjalankan logic tertentu.

Struktur Dasar:

```
Route::METHOD('/path', callback_atau_controller);
```

HTTP Methods:

- GET: Mengambil data
- POST: Mengirim data baru
- PUT/PATCH: Mengubah data
- DELETE: Menghapus data

Contoh Sederhana:

```
Route::get('/hello', function () {
    return 'Hello Laravel';
});

Route::post('/users', function () {
    return 'User created';
});
```

File Route:

- routes/web.php - untuk web routes
- routes/api.php - untuk API routes

Made with GAMMA

B. ROUTE PARAMETER - Static & Dynamic

Route Parameter memungkinkan URL dinamis yang dapat menerima nilai berbeda.

1. STATIC ROUTE (Rute Tetap)

Rute yang selalu sama, tidak ada parameter:

```
Route::get('/about', function () {
    return 'Halaman About';
});

Route::get('/contact', function () {
    return 'Halaman Contact';
});
```

2. DYNAMIC ROUTE (Rute Dinamis)

Rute yang dapat menerima parameter:

```
Route::get('/users/{id}', function ($id) {
    return "User ID: " . $id;
});
```

Contoh akses:

- /users/1 → User ID: 1
- /users/5 → User ID: 5

3. REQUIRED PARAMETER

Parameter yang harus ada:

```
Route::get('/products/{id}', [ProductController::class, 'show']);
```

4. OPTIONAL PARAMETER

Parameter yang boleh tidak ada:

```
Route::get('/posts/{id?}', function ($id = null) {
    if ($id) {
        return "Post ID: " . $id;
    }
    return "Semua posts";
});
```

5. MULTIPLE PARAMETER

```
Route::get('/users/{userId}/posts/{postId}', function ($userId, $postId) {
    return "User: $userId, Post: $postId";
});
```

Akses: /users/3/posts/7 → User: 3, Post: 7

Made with GAMMA



C. CONTROLLER - Membuat & Menghubungkan

Controller adalah class yang berisi logic aplikasi. Setiap method dalam controller menangani satu action.

1. MEMBUAT CONTROLLER

Gunakan artisan command:

```
php artisan make:controller ProductController
```

File akan dibuat di: app/Http/Controllers/ProductController.php

2. STRUKTUR CONTROLLER DASAR

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProductController extends Controller
{
    // Method untuk menampilkan list produk
    public function index()
    {
        return view('products.index');
    }

    // Method untuk menampilkan detail produk
    public function show($id)
    {
        return view('products.show', ['id' => $id]);
    }

    // Method untuk form create
    public function create()
    {
        return view('products.create');
    }

    // Method untuk menyimpan data
    public function store(Request $request)
    {
        // Proses penyimpanan data
        return redirect('/products');
    }
}
```

3. MENGHUBUNGKAN ROUTE DENGAN CONTROLLER

Syntax:

```
Route::METHOD('/path', [ControllerClass::class, 'methodName']);
```

Contoh:

```
Route::get('/products', [ProductController::class, 'index']);
Route::get('/products/{id}', [ProductController::class, 'show']);
Route::get('/products/create', [ProductController::class, 'create']);
Route::post('/products', [ProductController::class, 'store']);
```

4. RESOURCE ROUTE (Shortcut)

Untuk CRUD operations, gunakan:

```
Route::resource('products', ProductController::class);
```

Ini otomatis membuat 7 routes:

- GET /products → index
- GET /products/create → create
- POST /products → store
- GET /products/{id} → show
- GET /products/{id}/edit → edit
- PUT /products/{id} → update
- DELETE /products/{id} → destroy

Made with GAMMA

D. REQUEST & RESPONSE - Memahami Alur Data

REQUEST adalah data yang dikirim client ke server.

RESPONSE adalah data yang dikirim server kembali ke client.

1. APA ITU REQUEST?

Request berisi informasi dari client:

- **a) URL**
  - Menentukan resource mana yang diminta
  - Contoh: /products/5
- **b) HTTP Method**
  - GET, POST, PUT, DELETE, PATCH
  - Menentukan action apa yang dilakukan
- **c) Query Parameter**
  - Data di URL setelah ?
  - Contoh: /products?sort=name&page=2
  - Akses di controller: $request->query('sort')
- **d) Form Data (Body)**
  - Data dari form HTML
  - Dikirim via POST/PUT/PATCH
  - Akses di controller: $request->input('name')
- **e) Header**
  - Metadata tentang request
  - Contoh: Content-Type, Authorization
  - Akses: $request->header('Authorization')
- **f) File Upload**
  - File yang di-upload user
  - Akses: $request->file('photo')

2. MENGAKSES REQUEST DI CONTROLLER

```
public function store(Request $request)
{
    // Ambil satu input
    $name = $request->input('name');

    // Ambil semua input
    $data = $request->all();

    // Ambil query parameter
    $sort = $request->query('sort');

    // Ambil file
    $file = $request->file('photo');

    // Cek apakah input ada
    if ($request->has('email')) {
        // ...
    }
}
```

3. APA ITU RESPONSE?

Response adalah hasil yang dikirim server ke client.

Tipe Response:

- **a) HTML Response**

```
return view('products.index');
```

- **b) JSON Response**

```
return response()->json([
    'status' => 'success',
    'data' => $products
]);
```

- **c) Redirect Response**

```
return redirect('/products');
return redirect()->back();
```

- **d) Download Response**

```
return response()->download('file.pdf');
```

- **e) Plain Text Response**

```
return 'Hello World';
```

4. STATUS CODE

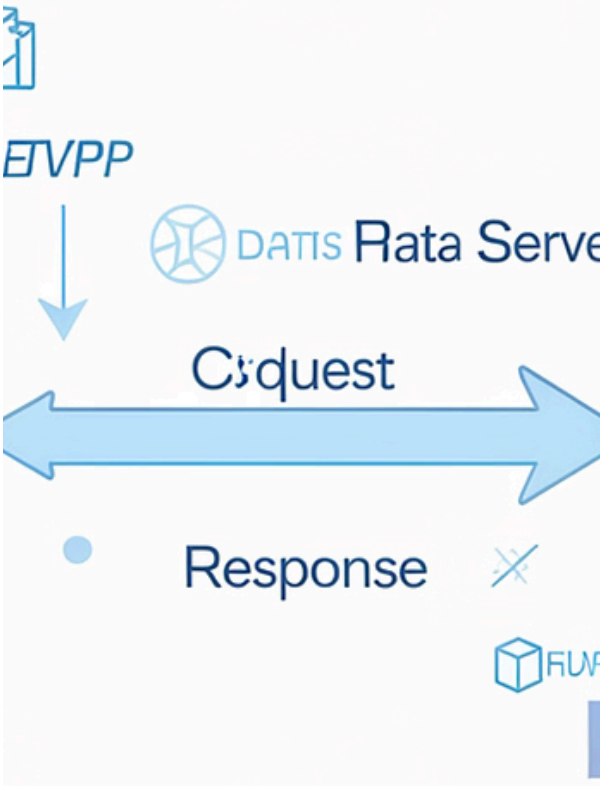
Status code menunjukkan hasil request:

- 200 OK - Request berhasil
- 201 Created - Resource berhasil dibuat
- 400 Bad Request - Request tidak valid
- 401 Unauthorized - Tidak terautentikasi
- 403 Forbidden - Tidak memiliki akses
- 404 Not Found - Resource tidak ditemukan
- 500 Internal Server Error - Error di server

Contoh:

```
return response()->json(['error' => 'Not found'], 404);
```

5. FLOW LENGKAP REQUEST-RESPONSE



1 **Browser mengirim request**

Made with GAMMA

E. BLADE TEMPLATE - Syntax & Fitur Dasar

Blade adalah template engine Laravel yang memudahkan menampilkan data dinamis di HTML.

1. STRUKTUR FOLDER BLADE

```
resources/
├── views/
│   ├── layouts/
│   │   └── app.blade.php
│   ├── products/
│   │   ├── index.blade.php
│   │   ├── show.blade.php
│   │   └── create.blade.php
│   └── welcome.blade.php
```

File Blade selalu berakhir dengan `.blade.php`

2. MENAMPILKAN DATA (Echo)

Syntax: `{{ $variable }}`

Contoh:

```
<h1>{{ $title }}</h1>
<p>{{ $description }}</p>
```

Dari controller:

```
return view('products.index', [
  'title' => 'Daftar Produk',
  'description' => 'Semua produk kami'
]);
```

3. CONDITIONAL (IF-ELSE)

1

If-Else Logic

Syntax: `@if`, `@elseif`, `@else`, `@endif`

```
@if ($user->isAdmin())
  <p>Anda adalah admin</p>
@elseif ($user->isModerator())
  <p>Anda adalah moderator</p>
@else
  <p>Anda adalah user biasa</p>
@endif
```

2

Unless Shortcut

Shortcut:

```
@unless ($user->isAdmin())
  <p>Anda bukan admin</p>
@endunless
```

4. LOOPING

a) @foreach

```
@foreach ($products as $product)
  <div>
    <h3>{{ $product->name }}</h3>
    <p>Rp {{ $product->price }}</p>
  </div>
@endforeach
```

b) @for

```
@for ($i = 0; $i < 10; $i++)
  <p>Nomor: {{ $i }}</p>
@endfor
```

c) @forelse

(foreach dengan fallback jika kosong)

```
@forelse ($products as $product)
  <p>{{ $product->name }}</p>
@empty
  <p>Tidak ada produk</p>
@endforelse
```

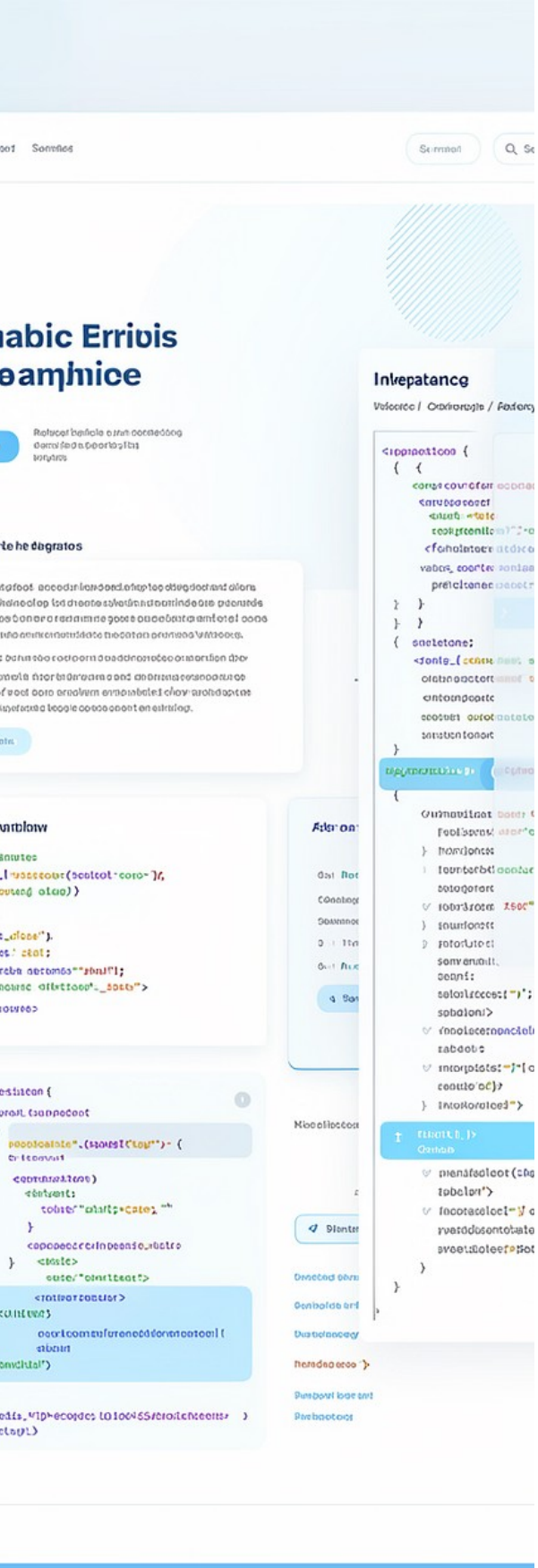
5. VARIABLE LOOP

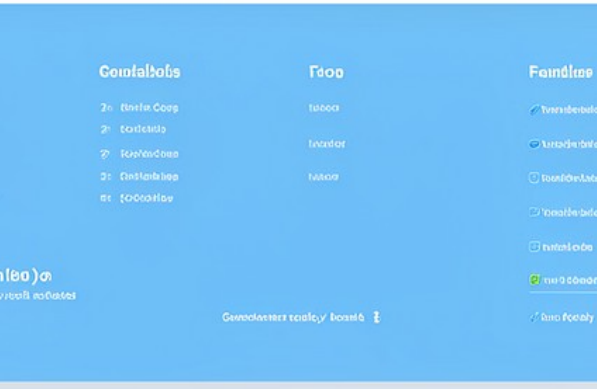
Di dalam `@foreach`, ada variable `$loop`:

```
@foreach ($products as $product)
  <p>
    Produk ke-{{ $loop->iteration }} dari {{ $loop->count }}
    @if ($loop->first)
      (Produk pertama)
    @endif
    @if ($loop->last)
```



Made with GAMMA





F. BLADE LAYOUT - Struktur & Inheritance

Layout adalah template master yang digunakan oleh multiple views. Ini mencegah pengulangan kode HTML.

1. MEMBUAT LAYOUT MASTER

File: `resources/views/layouts/app.blade.php`

Struktur dasar layout:

- DOCTYPE dan meta tags
- Header dengan navbar
- Main content area dengan `@yield('content')`
- Footer
- Script tags

Contoh `@yield` directives:

- `@yield('title')` untuk judul halaman
- `@yield('content')` untuk konten utama
- `@yield('extra-css')` untuk CSS tambahan
- `@yield('extra-js')` untuk JS tambahan

2. MENGGUNAKAN LAYOUT DI VIEW

File: `resources/views/products/index.blade.php`

Syntax:

- `@extends('layouts.app')` - Menggunakan layout app.blade.php
- `@section('title', 'Daftar Produk')` - Mengisi section title
- `@section('content')` ... `@endsection` - Mengisi section content

Contoh penggunaan:

- Menampilkan daftar produk dalam tabel
- Menggunakan `@foreach` untuk loop data
- Mengakses property dari object

3. MULTIPLE SECTIONS

Layout dapat memiliki multiple `@yield` untuk berbagai bagian:

- `@yield('extra-css')` di head
- `@yield('content')` di body
- `@yield('extra-js')` di akhir body

View dapat mengisi semua sections sesuai kebutuhan.

4. NESTED LAYOUT

- Layout dapat extend layout lain:
- `layouts/admin.blade.php` extends `layouts/app.blade.php`
- View menggunakan admin layout untuk halaman admin
- Memungkinkan struktur hierarki layout

5. KEUNTUNGAN MENGGUNAKAN LAYOUT

✓

Tidak mengulang HTML (header, footer, navbar)

Struktur halaman konsisten

Lebih mudah maintenance

Perubahan global hanya di satu tempat

Kode lebih rapi dan terorganisir

6. STRUKTUR FOLDER BEST PRACTICE

```
resources/views/
├── layouts/
│   ├── app.blade.php (layout utama)
│   ├── admin.blade.php (layout admin)
│   └── guest.blade.php (layout guest)
├── products/
│   ├── index.blade.php
│   ├── show.blade.php
│   ├── create.blade.php
│   └── edit.blade.php
├── users/
│   ├── index.blade.php
│   └── profile.blade.php
└── components/
    ├── header.blade.php
    ├── footer.blade.php
    ├── navbar.blade.php
    └── product-card.blade.php
```

Made with GAMMA

G. FORM LARAVEL - Handling Form Data

Form adalah cara user mengirim data ke server. Laravel menyediakan fitur keamanan untuk form.

1. MEMBUAT FORM HTML

Contoh form sederhana:

```
<form method="POST" action="/products">
  @csrf

  <label>Nama Produk</label>
  <input type="text" name="name" required>

  <label>Harga</label>
  <input type="number" name="price" required>

  <label>Deskripsi</label>
  <textarea name="description"></textarea>

  <button type="submit">Simpan</button>
</form>
```

Penjelasan:

- method="POST" - Mengirim data via POST
- action="/products" - Mengirim ke route /products
- @csrf - Token keamanan Laravel (WAJIB!)
- name attribute - Nama field yang akan diterima controller

2. CSRF TOKEN (@csrf)

CSRF = Cross-Site Request Forgery

@csrf adalah directive Blade yang menghasilkan hidden input dengan token keamanan.

Tanpa @csrf, form akan ditolak dengan error 419.

Contoh output:

```
<input type="hidden" name="_token" value="abc123...">
```

3. MENERIMA DATA DI CONTROLLER

```
public function store(Request $request)
{
  // Ambil satu input
  $name = $request->input('name');
  $price = $request->input('price');
  $description = $request->input('description');

  // Atau ambil semua input
  $data = $request->all();

  // Simpan ke database
  Product::create($data);

  // Redirect ke halaman lain
  return redirect('/products')->with('success', 'Produk berhasil ditambah');
}
```

4. FORM VALIDATION

Validasi memastikan data yang diterima valid:

```
public function store(Request $request)
{
  // Validasi input
  $validated = $request->validate([
    'name' => 'required|string|max:255',
    'price' => 'required|numeric|min:0',
    'description' => 'nullable|string'
  ]);

  // Jika validasi gagal, otomatis redirect ke form
  // dengan error messages

  // Jika validasi berhasil, lanjutkan
  Product::create($validated);

  return redirect('/products');
}
```

Validation rules:

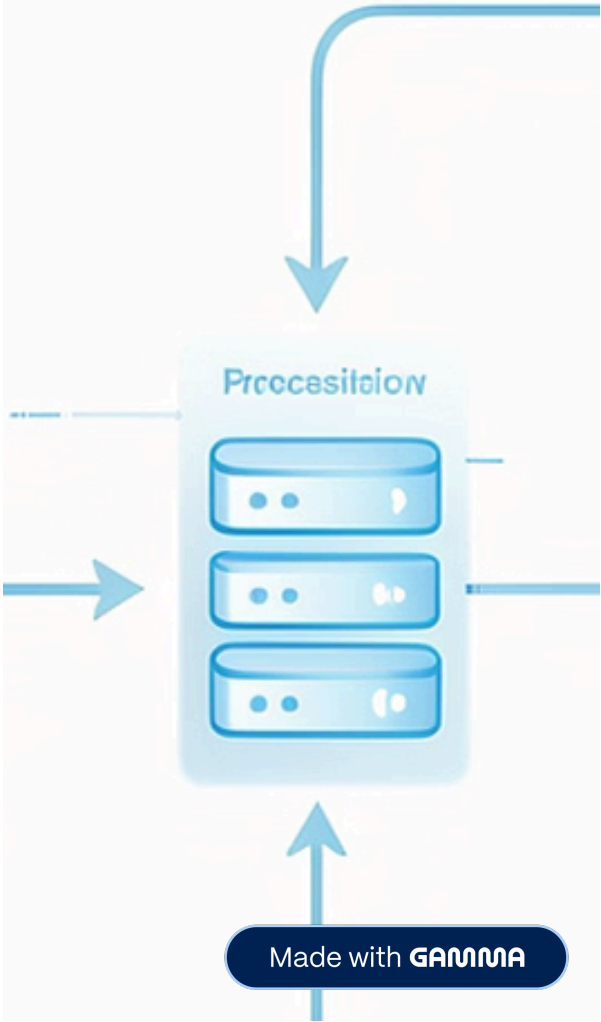
- required - Field harus diisi
- string - Harus string
- numeric - Harus angka
- email - Harus email valid
- max:255 - Maksimal 255 karakter
- min:0 - Minimal 0
- unique:products - Harus unik di tabel products
- nullable - Boleh kosong

5. MENAMPILKAN ERROR DI FORM

Di Blade template:

```
<form method="POST" action="/products">
  @csrf

  <div>
    <label>Nama Produk</label>
    <input type="text" name="name" value="{{ old('name') }}">
```



Made with GAMMA



H. AI ASSISTANCE - Debugging & Code Explanation

AI dapat membantu menjelaskan kode Laravel dan debugging error dengan cepat.

1. MENGGUNAKAN AI UNTUK MENJELASKAN KODE

Prompt yang baik:

> "Saya sedang belajar Laravel. Berikut adalah route dan controller saya:
>
> [TEMPEL KODE ROUTE]
> [TEMPEL KODE CONTROLLER]
>
> Jelaskan alur request dari browser sampai response. Tunjukkan bagian mana yang terhubung dan jelaskan alasannya. Jangan langsung menulis ulang semua kode."

Apa yang AI akan jelaskan:

- Bagaimana route menangkap request
- Bagaimana controller memproses data
- Bagaimana view menampilkan hasil
- Hubungan antara setiap komponen

2. MENGGUNAKAN AI UNTUK DEBUGGING

Prompt yang baik:

> "Saya memiliki error Laravel berikut:
>
> [TEMPEL ERROR MESSAGE]
>
> Berikut kode saya:
>
> [TEMPEL KODE YANG ERROR]
>
> Analisis kemungkinan penyebabnya. Berikan diagnosis terlebih dahulu. Setelah itu berikan solusi paling sederhana."

Contoh error yang sering:

- Class not found
- Method not found
- Undefined variable
- CSRF token mismatch
- Route not found

3. TIPS DEBUGGING DENGAN AI

✓ Sertakan error message lengkap

</> Sertakan kode yang relevan

Jelaskan apa yang ingin dicapai

Tanyakan diagnosis dulu sebelum solusi

Minta solusi paling sederhana

Jangan minta rewrite seluruh kode

4. CONTOH DEBUGGING CASE

Error: "Class 'App\Http\Controllers\ProductController' not found"

Kemungkinan penyebab:

- File controller belum dibuat
- Namespace salah
- File belum di-save
- Typo di nama class

Solusi:

- Pastikan file ProductController.php ada di app/Http/Controllers/
- Pastikan namespace benar: namespace App\Http\Controllers;
- Pastikan nama class sesuai: class ProductController extends Controller

5. CONTOH EXPLANATION CASE

Pertanyaan: "Bagaimana alur dari user klik link sampai halaman muncul?"

AI akan menjelaskan:

1. User klik link /products
2. Browser mengirim GET request ke /products
3. Route menangkap request dan memanggil ProductController@index
4. Controller mengambil data dari database
5. Controller return view dengan data
6. Blade template render HTML dengan data
7. HTML dikirim ke browser
8. Browser menampilkan halaman

6. COMMON LARAVEL ERRORS & SOLUTIONS

Error: "CSRF token mismatch"
Solusi: Tambahkan @csrf di form

Error: "Undefined variable"
Solusi: Pastikan variable dikirim dari controller dengan view()

Error: "Method not found"
Solusi: Pastikan method ada di controller dan nama sesuai

Error: "Route not found"
Solusi: Pastikan route terdaftar di routes/web.php

Error: "Column not found"
Solusi: Pastikan nama column sesuai dengan database

7. BEST PRACTICES SAAT CODING

- Buat route terlebih dahulu
- Buat controller dengan method yang sesuai
- Buat view untuk menampilkan data
- Test setiap step dengan browser
- Gunakan dd() untuk debug variable
- Baca error message dengan teliti
- Gunakan AI untuk menjelaskan jika bingung

Made with GAMMA

AKTIVITAS PRAKTIK - Simple Product Page

Peserta akan membuat aplikasi sederhana dengan fitur menampilkan daftar produk dan detail produk.

TUJUAN AKTIVITAS

Membuat aplikasi dengan:

- ✓ Route untuk list dan detail produk
- ✓ ProductController dengan method index dan show
- ✓ Blade template untuk menampilkan data
- ✓ Layout yang konsisten

FITUR YANG AKAN DIBUAT

1. /products - Menampilkan daftar semua produk
2. /products/{id} - Menampilkan detail satu produk
3. ProductController dengan 2 method
4. Blade layout dan views

LANGKAH-LANGKAH IMPLEMENTASI

STEP 1: Buat Controller

```
php artisan make:controller ProductController
```

STEP 2: Buat Routes (routes/web.php)

```
Route::get('/products', [ProductController::class, 'index']);
Route::get('/products/{id}', [ProductController::class, 'show']);
```

STEP 3: Buat Controller Methods

```
public function index()
{
    $products = [
        ['id' => 1, 'name' => 'Laptop', 'price' => 10000000],
        ['id' => 2, 'name' => 'Mouse', 'price' => 200000],
        ['id' => 3, 'name' => 'Keyboard', 'price' => 500000],
    ];

    return view('products.index', ['products' => $products]);
}

public function show($id)
{
    $products = [
        1 => ['id' => 1, 'name' => 'Laptop', 'price' => 10000000, 'desc' => 'Laptop gaming'],
        2 => ['id' => 2, 'name' => 'Mouse', 'price' => 200000, 'desc' => 'Mouse wireless'],
        3 => ['id' => 3, 'name' => 'Keyboard', 'price' => 500000, 'desc' => 'Keyboard mekanik'],
    ];

    $product = $products[$id] ?? null;

    if (!$product) {
        abort(404);
    }

    return view('products.show', ['product' => $product]);
}
```

STEP 4: Buat Layout (resources/views/layouts/app.blade.php)

Struktur HTML dasar dengan: - Header dan navbar - @yield('content') untuk konten - Footer - CSS styling

STEP 5: Buat View Index (resources/views/products/index.blade.php)

```
@extends('layouts.app')

@section('content')
```

Daftar Produk

@foreach ($products as $product) @endforeach

ID	Nama	Harga	Action
{{ $product['id'] }}	{{ $product['name'] }}	Rp {{ number_format($product['price']) }}	Lihat Detail

@endsection

STEP 6: Buat View Show (resources/views/products/show.blade.php)

```
@extends('layouts.app')

@section('content')
```

{{ $product['name'] }}

 ID: {{ $product['id'] }}
 Harga: Rp {{ number_format($product['price']) }}
 Deskripsi: {{ $product['desc'] }} Kembali ke Daftar @endsection

TESTING

1. Buka browser ke http://localhost:8000/products
2. Lihat daftar produk
3. Klik "Lihat Detail" untuk melihat detail produk

Made with GAMMA

RINGKASAN MATERI 2

Berikut adalah ringkasan lengkap konsep-konsep penting yang telah dipelajari.

KONSEP UTAMA

1. ROUTING

- Route menentukan URL mana yang akan menjalankan logic tertentu
- Syntax: Route::METHOD('/path', callback_atau_controller)
- HTTP Methods: GET, POST, PUT/PATCH, DELETE
- File: routes/web.php

2. ROUTE PARAMETER

- Static route: /about (selalu sama)
- Dynamic route: /users/{id} (dapat menerima parameter)
- Required parameter: {id} (harus ada)
- Optional parameter: {id?} (boleh tidak ada)
- Multiple parameter: /users/{userId}/posts/{postId}

3. CONTROLLER

- Class yang berisi logic aplikasi
- Setiap method menangani satu action
- Dibuat dengan: php artisan make:controller ProductController
- Dihubungkan ke route: [ProductController::class, 'methodName']
- Resource route: Route::resource('products', ProductController::class)

4. REQUEST & RESPONSE

- Request: data dari client (URL, method, form data, header, file)
- Response: hasil dari server (HTML, JSON, redirect, download)
- Status code: 200 (OK), 201 (Created), 404 (Not Found), 500 (Error)
- Akses request: $request->input(), $request->query(), $request->file()

5. BLADE TEMPLATE

- Template engine Laravel untuk menampilkan data dinamis
- Echo: {{ $variable }}
- Conditional: @if, @elseif, @else, @endif
- Looping: @foreach, @for, @forelse
- Include: @include('view.name')
- Comment: {{-- comment --}}

6. BLADE LAYOUT

- Layout master: @yield('section')
- Extend layout: @extends('layouts.app')
- Fill section: @section('content') ... @endsection
- Keuntungan: tidak mengulang HTML, konsisten, mudah maintenance

7. FORM LARAVEL

- Form HTML dengan method POST
- CSRF token: @csrf (WAJIB untuk keamanan)
- Validasi: $request->validate([...])
- Error handling: @error('field') ... @enderror
- Old value: old('field')

8. AI ASSISTANCE

- Menjelaskan kode dan alur request-response
- Debugging error dengan diagnosis terlebih dahulu
- Tips: sertakan error message, kode, dan tujuan
- Common errors: CSRF mismatch, undefined variable, route not found

ALUR REQUEST-RESPONSE LENGKAP

1. **Browser**
   (User klik link atau submit form)
2. **Route**
   (Route menangkap request)
3. **Controller**
   (Controller memproses logic)
4. **View (Blade Template)**
   (Template render HTML dengan data)
5. **Response**
   (HTML dikirim ke browser)
6. **Browser menampilkan halaman**

BEST PRACTICES

- **Buat route terlebih dahulu**
- **Buat controller dengan method yang sesuai**
- **Buat view untuk menampilkan data**
- **Gunakan layout untuk konsistensi**
- **Validasi semua input dari user**
- **Gunakan @csrf untuk form**
- **Test setiap step dengan browser**
- **Gunakan AI untuk debugging jika bingung**
- **Baca error message dengan teliti**
- **Gunakan dd() untuk debug variable**

FOLDER STRUCTURE

```
app/
├── Http/
│   └── Controllers/
│       └── ProductController.php

routes/
├── web.php
└── api.php

resources/
└── views/
    ├── layouts/
```

Made with GAMMA